Exp.no : 11 **KNN ALGORITHM**

AIM:

Experiment to understand KNN algorithm for a given data set

PYTHON CODE:

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report

# Read the dataset
df = pd.read_csv("Iris.csv")

# Display dataset information
df.info()

# Display number of samples in each variety
print(df.variety.value_counts())

# Display first 5 rows
print(df.head())

# Separate features and labels
features = df.iloc[:, :-1].values
label = df.iloc[:, 4].values

# Split data into training and testing sets
xtrain, xtest, ytrain, ytest = train_test_split(
    features,
    label,
    test_size=0.2,
    random_state=42
)

# Create KNN model with 5 neighbors
model_KNN = KNeighborsClassifier(n_neighbors=5)

# Train the model
model_KNN.fit(xtrain, ytrain)
```

```
# Training accuracy
print(model_KNN.score(xtrain, ytrain))

# Testing accuracy
print(model_KNN.score(xtest, ytest))

# Confusion matrix
print(confusion_matrix(label, model_KNN.predict(features)))

# Classification report
print(classification_report(label, model_KNN.predict(features)))
```

OUTPUT:

```
RangeIndex: 150 entries, 0 to 149

Data columns:
sepal.length
sepal.width
petal.length
petal.width
variety
   sepal.length  sepal.width  petal.length  petal.width  variety
0           5.1          3.5           1.4          0.2   Setosa
1           4.9          3.0           1.4          0.2   Setosa
2           4.7          3.2           1.3          0.2   Setosa
3           4.6          3.1           1.5          0.2   Setosa
4           5.0          3.6           1.4          0.2   Setosa
0.9583333333333334
1.0
[[50  0  0]
 [ 0 47  3]
 [ 0  2 48]]
              precision    recall  f1-score   support

Setosa           1.00      1.00      1.00        50
Versicolor       0.96      0.94      0.95        50
Virginica        0.94      0.96      0.95        50

accuracy                             0.97       150
macro avg        0.97      0.97      0.97       150
weighted avg     0.97      0.97      0.97       150
```

RESULT:

The code is executed successfully.